Oracle / PL/SQL.

- → GroupBy
- → Order By
- → With
- → Aggregate operations
- → Constraints
- → Indexes

} SQL - 6 topics.

SQL - 1970 develop ⟶ IBM Computer Scientists

SQL - Standard Query Language.

SQL Commands ——— Create, Select, Insert, Update, Delete and Drop.

DDL — Create, Alter, Drop

(Data Definition Language)

DML - Select, Insert, Update, delete

(Data Manipulation Language)

DCL - Grant, Revoke.

(Data Control Language)

RDBMS - Relational Data base Management System

Syntax :-

```
SELET  column1, column2, ... columnN
FROM   table_name;
```

Distinct clause :-

```
SELET  DISTINCT column1, column2, ... columnN
FROM   table_name;
```

Where / AND / OR clause :-

In Clause :-

```
SELET  column1, column2, ... columnN
FROM   table_name
WHERE  column_name IN (val-1, val-2, ..., val-N);
```

Between Clause :-

```
SELET  column1, column2, ... columnN
FROM   table_name
WHERE  column_name BETWEEN val-1 AND val-2;
```

LIKE Clause :

```
SELET   column1, column2 .      column-N

FROM    table name.
where   column - name  LIKE { PATTERN };
```

Order By clause :-

```
SELET   column1,  column2 .      columnN

FROM    table-name
where   CONDITION
ORDER   BY  column-name { ASC || DESC };
```

Group By clause :-

```
SELET   SUM (column-name)

FROM    table-name

WHERE   CONDITIOIN

GROUP BY   column-name;
```

SQL Constraints : (rules (or) restrictions)

- → Not Null constraints
- → UNIQUE
- → Primary Key
- → Foreign Key
- → Check
- → Default
- → Index constraint.

(i) Not Null constraints :-

→ Not Null constraints on Create Table.

```
CREATE TABLE employee (
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
ADDRESS varchar(20)
);
```

(ii) Unique :- (Create & Alter)

```
CREATE TABLE employee (
ID int(6) UNIQUE,
NAME varchar(10) NOT NULL,
ADDRESS varchar(20)
);
```

iii) PRIMARY KEY : (CREATE, ALTER)

PRIMARY KEY = NOT NULL + UNIQUE

```
CREATE TABLE employee (
ID int (6),
NAME varchar (10) NOT NULL,
ADDRESS varchar (20),
PRIMARY KEY (ID)
);
```

(or)

```
CREATE TABLE employee (
ID int (6),
Name varchar (10) NOT NULL,
ADDRESS varchar (20),
PRIMARY KEY (ID, NAME));
```

Ex: INSERT INTO employee (ID, NAME, ADDR
VALUES (1, 'NICK', 'MUMBAI');

(iv) FOREIGN KEY

ID	NAME	ADDRESS	DEPT_ID
1	Saloni	Mumbai	2
2	Nick	Pune	1

FOREIGN KE

Emp — Table

Child table

ID	DEPT_NAME
1	Admin
2	HR
3	Devevdpm

PRIMARY KEY.

Dep_Table

Parent table

Foriegn Key ⟶ child table
Primary Key ⟶ Parent table

Department table

```
CREATE TABLE department (
ID int(6) PRIMARY KEY,
DEPT_NAME varchar(10) NOT NULL
);
```

How to add Foreign Key constraint on a table?

```
CREATE TABLE employee(
 ID int(6) NOT NULL,
 NAME varchar(10) NOT NULL,
 ADDRESS varchar(20),
 DEPT_ID int(6),
 FOREIGN KEY (DEPT_ID) REFERENCES department(ID)
);
```

↑ primary key

(iv) CHECK:

```
CREATE TABLE employer(
   ID int(6),
   NAME varchar(10)  CHECK(NAME != 'Saloni'),
   ADDRESS varchar(20)
);
```

(vi) DEFAULT:

```
CREATE  TABLE employee(
    ID  int(6),
    NAME  varchar(30)  DEFAULT 'NEW USER',
    ADDRESS  Varchar(20)
);
```

GROUP BY CLAUSE:

Imp points:

→ GROUP BY CLAUSE → SELECT statement

→ After where clause the "Group By" clause will place

where → Group By → Having → order by

= Syntax:-

```
SELECT column_name, function_name(column-name)
FROM table-name
where condition
GROUP BY column_name 1, column_name 2
HAVING Condition
ORDER BY column_name 1, column_name 2
```

Use Cases of Group By

* GROUP BY with One Column
* GROUP BY with Two Columns
* GROUP BY and ORDER BY
* GROUP BY and HAVING
* GROUP BY, HAVING and WHERE

* GROUP BY With One Column

→ Find sum of salary of each department from employee_info table.

```
SELECT department, SUM(salary)
From employee_info
GROUP BY department;
```

* GROUP BY And ORDER BY

→ Find sum of salary of each department from employee_info table and also arrange the result in ascending order according to department.

```
SELET department, SUM(Salary)
From employee_info
GROUP BY department
ORDER BY department asc;
```

* GROUP BY And Having

-> Find sum of Salary of each department from employee_info table where the sum of salary is more than 17000.

```
SELET department, SUM(Salary)
From employee_info
GROUP BY department
HAVING SUM(salary) > 17000;
```

> = 17000 records will not fetch.

* GROUP BY, HAVING and WHERE

-> Find sum of salary of each department except 'Admin' from employee_info table where the sum of salary is more than 17000.

```
SELET department, SUM(Salary)
From employee_info
WHERE department != 'Admin'
GROUP BY department
HAVING SUM(salary) > 17000
```

GROUP BY with Two Columns

→ Find the number of studying same subject in same semester.

```
SELECT subject, semester, COUNT(*)
From student
GROUP BY subject, semester;
```

ORDER BY :- By default it will fetch Ascending.

```
SELET department, SUM(salary)
From employee_info
GROUP BY department
ORDER BY department asc;
```

→ Ascending

```
SELET department, SUM(salary)
From employee_info
GROUP BY department
ORDER BY department desc;
```

→ descending

Aggregate operations :-

total 5 functions

count()
sum()
avg()
min()
max()

1. Count Function :- numeric, non-numeric } It will work

→ COUNT(*) considers duplicate and Null.

Syntax:- COUNT(*)

(or)

COUNT([ALL|DISTINCT] expression)

i.
```
SELECT COUNT(*)
FROM PRODUCT_MAST;
```

ii.
```
SELECT COUNT(*)
FROM PRODUCT_MAST
WHERE RATE >= 20;
```

iii.
```
SELECT COUNT(DISTINCT COMPANY)
FROM PRODUCT_MAST;
```
→ Select with distinct

iv.
```
SELECT COMPANY, COUNT(*)
FROM PRODUCT_MAST
GROUP BY COMPANY;
```

v,
```
SELECT COMPANY, COUNT(*)
FROM PRODUCT_MAST
GROUP BY COMPANY
HAVING COUNT(*) > 2;
```

2, SUM FUNCTION :- Numerics

Syntax :

```
SUM()
(Q)
SUM([ALL|DISTINCT] expression)
```

i,
```
SELET SUM(COST)
FROM PRODUCT_MAST;
```

ii,
```
SELET SUM(COST)
FROM PRODUCT_MAST
WHERE QTY > 3;
```

iii,
```
SELECT SUM(COST)
FROM PRODUCT_MAST
WHEEE QTY > 3
GROUP BY COMPANY;
```

iv,
```
SELET COMPANY, SUM(COST)
FROM PRODUCT_MAST
GROUP BY COMPANY
HAVING SUM(COST) >= 170;
```

3. AVG FUNCTION : Numeric types

Syntax : AVG()

(or)

AVG([ALL | DISTINCT] expression)

Examples :

```
SELECT AVG(COST)
FROM PRODUCT_MAST
```

(4) MAX FUNCTION : Numeric

MAX()

(or)

MAX([ALL | DISTINCT] expression)

Example :

```
SETET MAX(RATE)
FROM PRODUCT_MAST;
```

(5) MIN FUNCTION : Numeric.

MIN()

(or)

MIN([ALL | DISTONCT] expression)

Example :

```
SELECT MIN(RATE)
FROM PRODUCT_MAST;
```

With Clause :

CTE - Common Table Expression

(or)

Sub-Query Factoring.

(or)

with clause

* whose salary is greater than the average salary of all the employee's in the table.

salary > average_salary_of_all_employee's

```
with alias () as
    (select avg(salary) from employee)
```

← Syntax

(alias → average_salary)

```
with average_salary (avg_sal) as
    (select cast(avg(salary) as int) from employee)

select *
from employee e, average_salary av
where e.salary > av.avg_sal;
```

(1) Total sales per each store .. total_sales

(2) Find the average sales with respect all the stores .. avg_sales

(3) Find the stores where total_sales > Avg_sales of all stores

(1) Total sales per each store -> total_sales

```
select s.store_id, sum(cost) as total_sales_per_store
from sales s
group by s.store_id;
```

(2) Find the average sales with respect all the stores avg_sales.

```
select cast(avg(total_sales_per_store) as int) as avg_Sales
from (select s.store_id, sum(cost) as total_sales_per_store
      from sales s
      group by s.store_id;)  x;
```

3) Find the stores where Total_Sales > Avg_Sales of all stores

```
Select *
from ( select s.store_id, sum(cost) as total_sales_per_store
       from sales s
       group by s.store_id) total_sales
join ( select cast(avg(total_sales_per_store) as int) as avg_sales
       from ( select s.store_id, sum(cost) as total_sales_per_store
              from sales s
              group by s.store_id) x ) avg_sales_of
on total_sales.total_sales_per_store > avg_sales_of.avg_sales;
```

↑
Complex & messy.
and get performance issues.

```
With Total_Sales (store_id, total_sales_per_store) as
     (select s.store_id, sum(cost) as total_sales_per_store
      from sales s
      group by s.store_id),
avg_sales (avg_sales_for_all_stores) as
     (select cast (avg(total_sales_per_store) as int) as avg_sales_for_all
      from Total_Sales)

Select *
from Total_Sales ts
join avg_sales av
on ts.total_sales_per_store > av.avg_sales_for_all ;
```